Documento Base de Estudio: Python Nivel 2

Guía de Referencia Estructurada: Daxus Curso | Entornos Virtuales y Programación Funcional

MÓDULO 1: Entornos Virtuales y Gestión de Dependencias

1. Conceptos Fundamentales

Un **entorno virtual** es un espacio de trabajo aislado dentro de un sistema diseñado para administrar el código de forma modular.

- **Propósito Principal:** Gestionar dependencias y mantener las bibliotecas independientes por cada proyecto.
- **Problema que Resuelve:** Evita conflictos de versiones cuando dos proyectos distintos requieren versiones diferentes de una misma biblioteca que chocarían en la instalación global.
- **Reproducibilidad:** Permite registrar las dependencias exactas, facilitando que otros desarrolladores recreen el mismo entorno de trabajo sin errores.

2. Gestión Tradicional (Módulo venv y pip)

El ecosistema estándar de Python utiliza herramientas nativas ejecutadas directamente en la terminal para aislar y nutrir un proyecto.

Concepto / Herramienta	Explicación / Comando (PowerShell)
Creación de Entorno	`python -m venv .venv` (Crea una carpeta aislada).
Activación	`.\.venv\Scripts\Activate.ps1` (Configura la sesión actual).
Desactivación	`deactivate` (Sale del entorno virtual en la sesión actual).
requirements.txt	Archivo de texto que enumera las dependencias del proyecto.
Instalación de Paquetes	`python -m pip install -r requirements.txt`
Control de Versión	`paquete==1.2.3` (Sintaxis para exigir una versión exacta).

3. Gestión Moderna (Poetry)

Poetry es una herramienta moderna y avanzada que unifica la creación del entorno y la gestión estricta de dependencias.

Acción	Comando de Poetry (Terminal)
Inicializar Proyecto	`poetry init`
Agregar Dependencia	`poetry add nombre_del_paquete`
Activar Entorno	`poetry shell`
Instalar Repo Clonado	`poetry install`
Salir / Desactivar	`exit` (o cerrar la ventana de la terminal)

MÓDULO 2: Estructuras Avanzadas y Programación Funcional

1. Comprehensions (Comprensiones)

Son técnicas que permiten construir colecciones de datos de manera altamente legible, matemáticamente elegante y en una sola línea de código.

- **List Comprehension:** Genera una nueva lista aplicando una expresión a cada elemento de un iterable.
- **Dict Comprehension:** Genera un diccionario mediante iteración, requiriendo siempre establecer pares `clave:valor`.
- **Sintaxis con Condicional:** `{clave: valor if condicion else not_condicion for elemento in iterable}`

2. Funciones Lambda

Conocidas como funciones anónimas (no declaradas mediante `def`), están diseñadas para ser evaluadas rápidamente en una única expresión.

- **Uso principal:** Se inyectan directamente como argumentos a otras funciones de orden superior.
- **Sintaxis Básica:** `lambda argumentos: expresion`
- **Anatomía:** Los *argumentos* son los parámetros de entrada; la *expresión* es la operación matemática/lógica cuyo resultado se devuelve de inmediato.

3. Funciones de Orden Superior (Higher-Order Functions)

Son funciones nativas del lenguaje que aceptan otras funciones (generalmente lambdas) como parámetros para procesar iterables de manera masiva.

Función	Propósito Principal	Sintaxis Base	Comportamiento / Retorno
map()	Aplicar una función a **todos** los elementos.	`map(funcion, iterable)`	Devuelve un objeto iterador (tipo map). Conserva la longitud original.
filter()	Filtrar elementos bajo una **condición**.	`filter(funcion, iterable)`	Devuelve un iterador solo con elementos que evalúan a `True`.
reduce()	Aislar el iterable a un **solo valor** acumulado.	`reduce(funcion, iter, [ini])`	Devuelve un valor único. Módulo requerido: `functools`.
sorted()	Ordenar elementos de manera **no destructiva**.	`sorted(iter, key=f)`	Devuelve una **nueva** lista ordenada. No altera el original.

Reglas Clave por Función

- **Reglas de map():** La función inyectada para operar puede ser *built-in* (ej. `int`), definida por el usuario o lambda.
- **Reglas de filter():** La función evaluadora *estrictamente* debe recibir un solo argumento y retornar un tipo booleano.
- **Reglas de reduce():** La función aplicada *debe* tomar exactamente dos argumentos (acumulador y elemento actual). El parámetro adicional `valor_inicial` es opcional y actúa como semilla base.
- **Reglas de sorted():** El parámetro opcional `key` requiere una función de un argumento para establecer la lógica de ordenamiento. Para invertir el resultado, se declara `reverse=True`.